	Batch: A-2 **Roll No.: 16010124046** **Experiment No.: 010** **Grade: AA / AB / BB / BC / CC / CD /DD** **Signature of the Staff In-charge with date**

Title: Study, Implementation, and Analysis of the Longest Common Subsequence Algorithm.

Objective: To compute longest common subsequence for the given two strings.

CO to be achieved:

CO 2	Analyze and solve problems for divide and conquer strategy, greedy method, dynamic programming approach and backtracking and branch & bound policies.
CO 3	Analyze and solve problems for different string matching algorithms.

Books/ Journals/ Websites referred:

1. **Ellis horowitz, Sarataj Sahni, S.Rajsekaran," Fundamentals of computer algorithm", University Press**
2. **T.H.Cormen ,C.E.Leiserson,R.L.Rivest and C.Stein," Introduction to algorithms",2nd Edition ,MIT press/McGraw Hill,2001**
3. **http://www.math.utah.edu/~alfeld/queens/queens.**

Pre Lab/ Prior Concepts:
Data structures, Concepts of algorithm analysis

Historical Profile:
Given 2 sequences, $X = x1 , ..., xm$ and $Y = y1 , ... , yn$, find a subsequence common to both whose length is longest. A subsequence doesn't have to be consecutive, but it has to be in order.

New Concepts to be learned:
String matching algorithm, Dynamic programming approach for LCS, Applications of LCS.

Recursive **Formulation:**

Define $c[i, j]$ = length of LCS of Xi and Yj .
Final answer will be computed with $c[m, n]$.

c[i, j]= 0
if i=0 or j=0.

$c[i, j] = c[i-1, j-1] + 1$
if $i,j > 0$ and $xi = yj$

$c[i, j] = \max(c[i-1, j], c[i, j-1])$
if $i, j > 0$ and $x_i <> y_j$

Algorithm: Longest Common Subsequence

Compute length of optimal solution-

```
LCS-LENGTH ( X , Y, m, n)
for i ← 1 to m
    do c[i, 0] ← 0
for j ← 0 to n
    do c[0, j ] ← 0
for i ← 1 to m
    do for j ← 1 to n
            do if xi = y j
             then c[i, j ] ← c[i − 1, j − 1] + 1
                  b[i, j ] ← "≈"
               else if c[i − 1, j ] ≥ c[i, j − 1]
                    then c[i, j ] ← c[i − 1, j ]
                         b[i, j ] ← "↑"
                    else c[i, j ] ← c[i, j − 1]
                         b[i, j ] ← "←"
return c and b
```

Print the solution-

PRINT-LCS*(b, X , i, j)*

```
if i = 0 or j = 0
  then return
if b[i, j ] = "≈"
  then PRINT-LCS(b, X , i − 1, j − 1)
       print xi
elseif b[i, j ] = "↑"
  then PRINT-LCS(b, X , i − 1, j )
else PRINT-LCS(b, X , i, j − 1)
```

Initial call is PRINT-LCS*(b, X , m, n)*.

$b[i, j]$ points to table entry whose subproblem we used in solving LCS of Xi and Yj.

When $b[i, j] = \approx$, we have extended LCS by one character. So longest com- mon subsequence = entries with $\approx$ in them.

Example: LCS computation

S1 = "Sharvil" 7+10
S2 = "Shravya"

	S	h	a	r	r	v	i	l
S	0	0	0	0	0	0	0	0
h	0	1→	1→	1→	1→	1→	1→	1←
a	0	1↑	1↑	1↑	1↑	1↑	1↑	1↑
r	0	1↑	1↑	1↑	1↑	1↑	1↑	1↑
v	0	1	2←	2←	2←	3←	3←	3←
i	0	1↑	D↖	2←	2←	3←	3←	3←
s	0	1↑	1↑	D↖	3←	3←	3←	3←
r	0	1↑	1↑	1↑	D↖	3←	2←	3←
y	0	1↑	1↑	1↑	1↑	D↖	4←	3←
a	0	1↑	1↑	1↑	1↑	4↑	4↖	4←
								4←

⇒ Hence, LCS = 4

Analysis of LCS computation

Analysis –

⇒ LCS computation uses nested for loops (length/size of the respective inputs].

⇒ ⇒ Hence, if n = size of string 1,
& m = size of string 2,
then,
time complexity will be $O(n \cdot m)$,
that is $O(n^2)$]

⇒ As we make a 2-D vector, the time complexity

$\Rightarrow O(n^2) \Rightarrow O(n+(n+11) \Rightarrow O(n^2)$.

Output:

```
PS C:\Users\jayth> cd "c:\Users\jayth\OneDrive\Desktop\" ; if ($?
Enter the value of s1: JAYTHACKER
Enter the value of s2: HACKER
Length of LCS: 6
PS C:\Users\jayth\OneDrive\Desktop> cd "c:\Users\jayth\OneDrive\D
Enter the value of s1: COMPUTER
Enter the value of s2: IMPUTER
Length of LCS: 6
PS C:\Users\jayth\OneDrive\Desktop> cd "c:\Users\jayth\OneDrive\D
Enter the value of s1: GenAptiTestExam
Enter the value of s2: GATE
Length of LCS: 4
PS C:\Users\jayth\OneDrive\Desktop>
```

Algorithm:

```
#include<bits/stdc++.h>
using namespace std;
int LCS(string s1, string s2) {
    int n = s1.length();
    int m = s2.length();
    vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
    for(int i = 1; i<=n; i++) {
        for(int j = 1; j<=m; j++) {
            if(s1[i-1] == s2[j-1]) {
                dp[i][j] = 1 + dp[i-1][j-1];
            } else {
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }
    return dp[n][m];
}
int main() {
    string s1, s2;
    cout << "Enter the value of s1: ";
    cin >> s1;
```

```
    cout << "Enter the value of s2: ";
    cin >> s2;
    cout << "Length of LCS: " << LCS(s1, s2) << "\n";
    return 0;
}
```

CONCLUSION:

From this experiment, we learned and implemented LCS (Least Common Sequence) using Dynamic Programming. This algorithm has real life applications like DNA matching.